

# Effectful Computations with Future Conditions: A Monadic Approach to Temporal Specification

AUTHOR NAME, Institution, Country

Pre- and post-conditions are the standard vocabulary for modular program specification, yet they are silent about obligations that span multiple calls. In real Haskell code such obligations are ubiquitous: a successful TLS handshake must eventually be followed by `bye` before the context is discarded; a taken `MVar` must eventually be put back on *every* code path; a `beginTx` must eventually reach `commit` or `rollback`; a submitted job must complete with probability at least  $p$ . No single pre/post pair can express these properties on its own, because the balancing operation lies in an unknown future context.

Building directly on Song et al. [2025]’s proposal of future conditions as a first-class extension to pre/post contracts, we present *Pledge*, a Haskell library that realises this idea as a monad: annotations that propagate obligations forward until they are discharged. The `Pledge` monad carries a data-dependent field `future :: a → eff`, allowing obligations to name the specific resource handle an operation returns. Monadic `bind` partially discharges the first computation’s future condition against the continuation’s postcondition via *subtraction*; the residual is conjoined with the continuation’s own future condition. *Pledge* is a *specification* library: it computes and reports residual obligations on pure annotation values, it does not instrument or guard a running IO computation, and the accompanying monad-law proof covers the *(ret, post, future)* triple, not the *pre* field, which is carried as a separate contract-checking annotation.

The design is parameterised over a Composable algebra with concatenation ( $\circ$ ), conjunction ( $\wedge$ ), subtraction ( $\setminus$ ), and identities `empty` and `universe`. We present four instances: trace protocols via extended regular expressions (RE, with Brzozowski-derivative subtraction and a direct  $LTL_f$  embedding); Presburger-guarded traces (GRE, discharged by `Z3`); semiring-weighted obligations (WRE, covering probabilistic and min-cost scenarios); and separation-logic heap predicates (SL, with magic wand as subtraction). All four share a single `bind` formula, demonstrating that trace-based, arithmetic, quantitative, and heap-ownership obligations are uniformly expressible within one algebraic interface, and we give a soundness theorem for the RE instance connecting a fully-discharged residual to satisfaction of the obligation by every terminating trace.

CCS Concepts: • **Software and its engineering** → **Functional languages**; *Data types and structures*; *Software verification and validation*.

Additional Key Words and Phrases: temporal specification, monadic effects, regular expressions, complement, Brzozowski derivatives, Presburger arithmetic, semiring weights, separation logic, resource lifecycle, Haskell

## ACM Reference Format:

Author Name. 2026. Effectful Computations with Future Conditions: A Monadic Approach to Temporal Specification. *Proc. ACM Program. Lang.* 1, HASKELL (July 2026), 30 pages.

## 1 Introduction

Pre- and post-conditions are the workhorse of modular program specification. A pre-condition constrains the state in which an operation may be invoked; a post-condition describes what holds

---

Author’s Contact Information: Author Name, Institution, City, Country, [author@institution.edu](mailto:author@institution.edu).

---

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 2475-1421/2026/7-ART

<https://doi.org/>

immediately after it returns. Yet a great many correctness properties are not local to a single call boundary: they *span* a sequence of operations, with arbitrary computation in between.

Consider three scenarios from the Haskell ecosystem: i) *TLS lifecycle*. A successful handshake must eventually be followed by `bye` before the context is discarded. A post-condition on handshake records success, but cannot enforce that `bye` will be called, because the discharge point lies in an unknown future context. The bracket combinator sidesteps this by lexically scoping the context, but cannot express richer message-exchange obligations or handle contexts passed across function boundaries. ii) *MVar discipline*. The `takeMVar` operation removes a value, leaving the variable empty; the caller must restore it with `putMVar` on every exit path. A post-condition records the empty state but cannot require that a future `putMVar` on the *same* variable will occur. iii) *Database savepoints*. The `transactionSave` function opens a fresh transaction that must itself be committed or rolled back; pre/post-conditions on the call cannot carry this obligation forward.

*Existing remedies and their limits*. The bracket function lexically scopes cleanup but cannot propagate obligations beyond the lexical scope. Both `ResourceT` [Snoyman 2011] and `Acquire/Managed` thread cleanup actions through every `bind` but model only the cleanup half of an obligation, discharging it uniformly at scope exit rather than by application-specific operations. Linear Haskell [Bernardy et al. 2018] enforces “consume exactly once” but cannot express branching discharge obligations or quantitative commitments. Parameterised and indexed monads [Atkey 2009] encode protocol states at the type level but require `RebindableSyntax` or `QualifiedDo`, produce opaque type errors, and are confined to a single domain per design.

*Relation to prior future-condition proposals*. The idea of a future condition itself is not new here: Song et al. [2025] introduce future conditions as a first-class extension to pre/post-condition contracts, formalising the notion that an operation may discharge temporal obligations onto its continuation, and Song et al. [2024] apply regular-expression-encoded temporal properties to guide automated program repair in `ProveNFix`. This paper’s contribution is not the concept of a future condition but a *library realisation* of it: a concrete `Pledge` monad, a derivative-based subtraction mechanism that implements the discharge relation algorithmically rather than declaratively, and four `Composable` instances — traces, Presburger-guarded traces, semiring-weighted traces, and separation-logic heaps — that the formal proposal of Song et al. [2025] leaves uninstantiated. We return to a detailed comparison in §10.

*Specification, not enforcement*. Before going further, we state plainly what `Pledge` is and is not, since this distinction matters for every example that follows. `Pledge` is a *specification* library: a `Pledge` value is a pure Haskell record of annotations (pre, post, future) that describes what a computation should do. Composing `Pledge` values via  $\gg$  computes a residual annotation; a programmer or a separate tool inspects that residual *before, or independently of*, running any real effect. `Pledge` does not instrument IO, does not intercept asynchronous exceptions, and does not by itself guarantee that a real running program obeys the obligations its specification names — it tells you, given a specification term, whether the obligations it encodes are met by every trace consistent with that term. §3.5 discusses concretely how a `Pledge` specification is intended to relate to a real effectful computation.

*Intended use*. Across our examples, a `Pledge`-annotated primitive (`tlsHandshake`, `takeMVar`, `malloc`) is written once, by the author of a library or an API, exactly as a Hoare-style pre/post-condition would be. The *consumer* of a library is the programmer composing these primitives with  $\gg$ ; that programmer’s obligation is only to inspect the residual pre and future of the composed term, not to write temporal specifications themselves. This division of labour matches ordinary contract-based programming: application programmers consume contracts, library authors write them. The

four Composable instances play different roles in this picture. RE is the primary, end-to-end usable instance: subtraction is fully computed by derivatives, and a residual of  $\neg\emptyset$  is a decidable certificate that every trace-based obligation is met (Theorem 4.3). GRE is equally usable, discharging the arithmetic component via an SMT call. WRE is usable for the quantitative obligations it targets (probability, cost), though, as §6 discusses, it deliberately gives up the complement operator and hence the  $LTL_f$  embedding. SL, by contrast, is included primarily as evidence that the Composable interface generalises beyond trace-shaped obligations to heap ownership: as §7 makes explicit, its subtraction records the magic wand *symbolically* and does not invoke a solver, so a non-universe SL residual identifies a candidate obligation for a programmer or an external tool (e.g. Smallfoot or Z3) to check, rather than a machine-checked verdict on its own.

*Our approach.* We propose the Pledge monad as a lightweight, compositional mechanism for specifying such obligations, requiring no language extensions beyond standard Haskell type classes. A value of type `Pledge eff a` carries three annotations: a precondition `pre :: eff`, a postcondition `post :: eff`, and a data-dependent future condition `future :: a → eff`. Because `future` ranges over the return value, an obligation can name the actual resource handle produced at runtime: `tlsHandshake` can declare that *whichever* Context it returns must eventually receive `bye`. Monadic `bind` partially discharges the first computation’s future via subtraction against the continuation’s postcondition; the residual is conjoined with the continuation’s own future condition. The user annotates individual operations only; the monad propagates obligations automatically.

The Composable typeclass abstracts the specification algebra into five operations: concatenation ( $\diamond$ ), conjunction ( $\wedge$ ), subtraction ( $\setminus$ ), and their respective identities `empty` and `universe`. It admits four independent instances:

- (*Trace protocol*, RE) Extended regular expressions over events, with complement as a first-class constructor. Subtraction is implemented via Brzowski derivatives;  $LTL_f$  operators embed directly as RE terms without automaton construction. This instance handles `takeMVar/putMVar` discipline, TLS handshake/bye pairing, transaction lifecycle, and lock protocols.
- (*Arithmetic bounds*, GRE) Each RE is paired with a Presburger arithmetic predicate over program variables, checked by an SMT solver (Z3 via SBV). A single annotation simultaneously enforces a protocol ordering *and* a numeric bound, catching, for example, counter overflow before any execution takes place.
- (*Quantitative obligation*, WRE) Boolean membership is generalised to a semiring weight. The probability semiring captures obligations of the form “this job completes with probability at least  $p$ ”; the tropical semiring gives minimum-cost scheduling obligations. A weight of `szero` in the residual signals an unfulfilled quantitative commitment.
- (*Heap ownership*, SL) Separation-logic heap predicates, with separating conjunction as sequential composition and magic wand as subtraction. Future conditions describe what heap state must hold when the computation completes; a leaked `Cell` in the residual `post` names the unreleased ownership precisely.

*Central insight.* The unifying mechanism is *Brzowski-derivative subtraction as obligation propagation*. When a computation  $e$  is sequenced with a continuation  $f$ , the continuation’s postcondition  $post(e_2)$  is subtracted from  $e$ ’s pending future obligation  $future(e)$  using the derivative: for each event  $a$  that  $post(e_2)$  can begin with,  $\partial_a(post(e_2)) \setminus \partial_a(future(e))$  advances both sides in lock-step, consuming the obligation event by event. The residual is what remains unmet. The insight is that this derivative-based subtraction, originally introduced for regular-language parsing, doubles as a compositional discharge mechanism for temporal obligations in a monadic setting. The four Composable instances are four realisations of the same pattern: RE subtraction via derivatives; GRE

subtraction via derivatives with a Presburger conjunction; WRE subtraction with semiring weights; and SL subtraction via the magic wand. All four share a single bind formula, and the diagnostic residual is always the same: anything not equal to universe names an unmet obligation precisely, and for the RE instance this diagnostic is backed by the soundness theorem of §4.1.

### Contributions.

- (1) We define the Pledge monad (§3), parameterised over a Composable algebra, with  $\text{pre} :: \text{eff}$ ,  $\text{post} :: \text{eff}$ , and a data-dependent future  $:: a \rightarrow \text{eff}$  field, explain how this pure specification layer relates to real effectful code (§3.5), and verify that the monad laws hold for the  $(\text{ret}, \text{post}, \text{future})$  triple (§9).
- (2) We instantiate the algebra to extended regular expressions over events with complement, using Brzozowski derivatives to implement subtraction (§2.1), show that  $\text{LTL}_f$  operators embed directly as RE terms (§2.3), and prove a soundness theorem connecting a discharged residual to satisfaction of the obligation on every terminating trace (§4.1).
- (3) We extend the RE instance to a guarded form (GRE, §5), pairing each RE with a Presburger arithmetic predicate over program variables, evaluated by an SMT solver (Z3 via SBV) at membership-check time.
- (4) We provide a weighted RE instance (WRE, §6), parameterised over a semiring, that generalises Boolean membership to a quantitative weight, recovering probabilistic and min-cost future conditions as special cases.
- (5) We provide a fourth, independent instantiation to separation-logic heap predicates (§7), demonstrating that the Composable interface extends beyond trace-based specifications, while being explicit that this instance records obligations symbolically rather than discharging them against a solver.
- (6) We demonstrate the approach on use cases across all four instances (§8), showing that resource leaks, ordering violations, arithmetic overflows, probabilistic obligations, and heap-ownership errors all surface as non-trivial residual annotations, favouring examples with non-trivial future conditions throughout.

## 2 Background

### 2.1 Extended Regular Expressions over Events

Events are the atomic units of observation; each carries a name and a list of typed arguments.

```
data Term = Str String | Num Int | List [Term]
data Event = Atom String [Term] -- concrete event, e.g. Atom "send" [Num 1]
           | Wildcard           -- pattern matching any single event
data RE = Bot | Epsilon | Single Event
        | Seq RE RE | Or RE RE | And RE RE | Star RE | Not RE
```

The specification language is an extended regular expression type that adds complement as a first-class constructor:

$$r ::= \emptyset \mid \varepsilon \mid e \mid r_1 \cdot r_2 \mid r_1 \vee r_2 \mid r_1 \wedge r_2 \mid r^* \mid \neg r$$

The constructors are standard. The empty language  $\emptyset$  (**Bot**) accepts no trace. The empty-trace language  $\varepsilon$  (**Epsilon**) accepts only the empty sequence. A single-event term  $e$  (**Single Event**) matches exactly one event; **Wildcard** is a pattern that matches any single event. Concatenation  $r_1 \cdot r_2$  (**Seq**) accepts a trace if it splits into a prefix in  $L(r_1)$  followed by a suffix in  $L(r_2)$ . Union  $r_1 \vee r_2$  (**Or**) accepts traces in either language. The Kleene star  $r^*$  (**Star**) accepts zero or more repetitions of traces from  $L(r)$ ;  $\Sigma^*$  is the set of all finite event sequences over the alphabet  $\Sigma$  (the universal language).

Intersection  $r_1 \wedge r_2$  (**And**) could be derived as  $\neg(\neg r_1 \vee \neg r_2)$  via De Morgan; we instead include **And** as a primitive constructor (matching the code in §2.1) so that its derivative can be computed directly, without paying for three extra **Not** nodes on every conjunction. Complement  $\neg r$  (**Not**) is the set of all traces in  $\Sigma^*$  *not* in  $L(r)$ . Moreover, we use the following five derived forms throughout:

|                                                                                               |                                      |
|-----------------------------------------------------------------------------------------------|--------------------------------------|
| anything $\triangleq \neg\emptyset$                                                           | (the universal language $\Sigma^*$ ) |
| finally( $e$ ) $\triangleq$ anything $\cdot e \cdot$ anything                                 | ( $e$ must occur eventually)         |
| globally( $e$ ) $\triangleq e^*$                                                              | ( $e$ at every step)                 |
| never( $e$ ) $\triangleq \neg$ finally( $e$ )                                                 | ( $e$ must never occur)              |
| noUntil( $e, g$ ) $\triangleq \neg((\Sigma \setminus \{g\})^* \cdot e \cdot \text{anything})$ | ( $e$ must not occur before $g$ )    |

noUntil( $e, g$ ) is nullable (the empty continuation satisfies it) and its derivative with respect to  $g$  is  $\Sigma^*$ , meaning once  $g$  occurs the restriction on  $e$  is lifted. This is the key combinator for *conditional* safety:  $e$  is forbidden until  $g$  re-enables it.

## 2.2 Algebraic Complement and Brzowski Derivatives

The Brzowski derivative [Brzowski 1964]  $\partial_a(r)$  computes the language of continuations after reading event  $a$ . The key law for complement is:

$$\partial_a(\neg r) = \neg(\partial_a(r)) \quad (1)$$

The nullability function  $\nu(r)$  (= True iff  $\varepsilon \in L(r)$ ) satisfies  $\nu(\neg r) = \neg \nu(r)$ . Together, equations (1) and the De Morgan laws

$$\neg(r_1 \vee r_2) = \neg r_1 \wedge \neg r_2 \quad \neg(r_1 \wedge r_2) = \neg r_1 \vee \neg r_2$$

allow complement to be propagated and simplified *purely algebraically*, without building a DFA. The normaliser additionally applies  $\neg\neg r = r$ ,  $\neg\emptyset = \emptyset$ , and  $\neg\neg\emptyset = \emptyset$ , and the same normaliser is reused to simplify **Not** nodes arising from **never** and **noUntil**. We define the nullable and derivative functions in Haskell as follows:

```

nullable :: RE → Bool
nullable (Not r)      = not (nullable r)      -- nu(Not r) = not(nu(r))
nullable (Star _)     = True
nullable (Seq r1 r2)  = nullable r1 && nullable r2
nullable (Or r1 r2)   = nullable r1 || nullable r2
nullable (And r1 r2)  = nullable r1 && nullable r2
nullable Epsilon      = True
nullable _            = False

derivative :: Event → RE → RE
derivative e (Not r)   = Not (derivative e r)  -- d_a(Not r) = Not(d_a(r))
derivative e (Single p) = if matchEvent e p then Epsilon else Bot
derivative e (Seq r1 r2)
  | nullable r1 = Or (Seq (derivative e r1) r2) (derivative e r2)
  | otherwise  = Seq (derivative e r1) r2
derivative e (Or r1 r2) = Or (derivative e r1) (derivative e r2)
derivative e (And r1 r2) = And (derivative e r1) (derivative e r2)

```

```

derivative e (Star r)      = Seq (derivative e r) (Star r)
derivative _ _            = Bot

```

The `firstWith` function returns the set of events (drawn from a supplied finite alphabet  $\Sigma_0$ ) that can begin a word in  $L(r)$ , driving the subtraction algorithm; first in the prose below always refers to this same function, applied to the alphabet of the expression itself (first  $r = \text{firstWith}(\text{atoms } r) r$ ). The complement case requires particular care. Naively writing `firstWith alph (Not r) = firstWith alph r` is *unsound*:  $L(\neg r)$  can begin with events *not* in  $\text{first}(r)$ . The correct characterisation is:

$$e \in \text{first}(\neg r, \Sigma_0) \iff e \in \Sigma_0 \wedge [\partial_e(r)] \neq \neg \emptyset \quad (2)$$

That is,  $e$  can open a word in  $L(\neg r)$  if and only if, after consuming  $e$ , the residual  $\partial_e(r)$  does *not* cover the entire language: there exists some continuation that remains outside  $L(r)$ . The condition  $[\partial_e(r)] \neq \neg \emptyset$  checks exactly this. If the normalised derivative equals the universal language  $\Sigma^*$ , then every continuation after  $e$  belongs to  $L(r)$ ; consequently, no continuation after  $e$  belongs to  $L(\neg r)$ , and  $e$  cannot open any word in that language.

```

atoms :: RE → [Event]
atoms (Single Wildcard) = []
atoms (Single e)        = [e]
atoms (Seq r1 r2)       = atoms r1 `union` atoms r2
atoms (Or r1 r2)        = atoms r1 `union` atoms r2
atoms (And r1 r2)       = atoms r1 `union` atoms r2
atoms (Star r)          = atoms r
atoms (Not r)           = atoms r
atoms _                 = []

firstWith :: [Event] → RE → [Event]
firstWith _ Bot         = []
firstWith _ Epsilon    = []
firstWith _ (Single e) = [e]
firstWith alph (Seq r1 r2)
  | nullable r1          = firstWith alph r1 `union` firstWith alph r2
  | otherwise            = firstWith alph r1
firstWith alph (Or r1 r2) = firstWith alph r1 `union` firstWith alph r2
firstWith alph (And r1 r2) = [e | e <- firstWith alph r1,
                                   e `elem` firstWith alph r2]
firstWith alph (Star r)  = firstWith alph r
firstWith alph (Not r)   = [e | e <- alph,
                               not (isTotal (normalize (derivative e r)))]
  where isTotal (Not Bot) = True; isTotal _ = False

```

The alphabet  $\Sigma_0$  is collected from an RE via `atoms`, which recursively gathers all concrete (**Atom**) events, excluding **Wildcard**. Inside subtraction (§4), both operands contribute to the alphabet, so that events appearing only under a complement on one side are still enumerated correctly by Equation (2).

*Two worked cases.* Two small examples fix the intuition for Equation (2). When  $r = a \cdot \Sigma^*$  and  $\Sigma_0 = \{a, b\}$ , the naive rule would return  $\{a\}$ , but  $L(\neg r)$  consists of words beginning with events

other than  $a$ :  $\partial_a(a \cdot \Sigma^*)$  normalises to  $\neg\emptyset$ , excluding  $a$ , while  $\partial_b(a \cdot \Sigma^*) = \emptyset \neq \neg\emptyset$  includes  $b$ , giving the correct  $\{b\}$ . When  $r = \varepsilon$  and  $\Sigma_0 = \{a, b\}$ , the naive rule returns  $\emptyset$  (since  $\text{first}(\varepsilon) = \emptyset$ ), but  $\neg\varepsilon$  accepts all non-empty traces:  $\partial_a(\varepsilon) = \emptyset \neq \neg\emptyset$  includes  $a$ , and likewise  $b$ , giving  $\{a, b\}$ . Table 1 extends these to seven further patterns. The first four rows confirm that complement does not exclude any alphabet event when the derivative is non-total. The double-negation row shows correct restoration of the original first set:  $\partial_b(\neg a) = \Sigma^*$  is total, so  $b$  is filtered out. In  $\neg(a \cdot \Sigma^*)$ ,  $\partial_a(a \cdot \Sigma^*)$  normalises to  $\Sigma^*$ , excluding  $a$ ; only  $b$  survives. Similarly,  $\neg(a \cdot \Sigma^* \vee b \cdot \Sigma^*)$  excludes both  $a$  and  $b$ , leaving only  $c$ , while the De Morgan row verifies that  $\neg(\neg a \wedge \neg b) = a \vee b$  yields  $\{a, b\}$ .

### 2.3 Embedding LTL<sub>f</sub>

Because complement is a first-class constructor, the standard LTL<sub>f</sub> operators translate directly into RE, with no automaton construction needed. Table 2 gives the full mapping.

The complement operator does the heavy lifting:  $G\phi \triangleq \neg F\neg\phi$  unfolds to  $\neg(\neg\emptyset \cdot \neg\llbracket\phi\rrbracket)$  using two applications of  $\neg$ ;  $F\phi$  and  $X\phi$  need only concatenation. The  $U$  case uses  $\text{step}(\phi)$ , the length-1 RE for a single event satisfying  $\phi$ , which is defined only when  $\phi$  is propositional.

## 3 The Pledge Monad

### 3.1 The Composable Class

We abstract the specification algebra via a type class:

```
class Composable a where
  concatenation :: a → a → a    -- (◊), unit: empty
  conjunction  :: a → a → a    -- (∧), unit: universe
  empty        :: a             -- identity for ◊ (varepsilon)
  universe     :: a             -- identity for ∧ (neg-emptyset)
  subtraction  :: a → a → a    -- (∖)
```

The five operations form exactly the interface required by the bind formula. Here concatenation sequences effects; conjunction combines simultaneous constraints. The identities `empty` and `universe` appear in pure. In particular, the operation `subtraction r1 r2` computes the residual of the *second* argument with respect to the first: it asks “which part of the obligation  $r_2$  is not yet discharged, given that a trace matching  $r_1$  has been observed?” The first argument  $r_1$  is the *observed* trace (a postcondition); the second argument  $r_2$  is the *pending obligation* (a precondition or future condition to be discharged). Two base cases fix the endpoints:  $\varepsilon \setminus r_2 = r_2$  (nothing observed, the full obligation  $r_2$  remains) and  $\Sigma^* \setminus r_2 = \varepsilon$  (everything observed, the obligation is fully discharged). The general

Table 1. Selected `firstWith` evaluations with **Not**.  $\Sigma_0$  is the explicitly supplied alphabet. Rows marked  $\dagger$  produce a *strict* subset of  $\Sigma_0$ .

| RE $r$                                   | $\Sigma_0$    | <code>firstWith</code> $\Sigma_0$ ( $\neg r$ ) | Rationale                                                                                |
|------------------------------------------|---------------|------------------------------------------------|------------------------------------------------------------------------------------------|
| $a$                                      | $\{a, b\}$    | $\{a, b\}$                                     | $\partial_a(a) = \varepsilon$ , $\partial_b(a) = \emptyset$ ; neither $\Sigma^*$         |
| $a \cdot b$                              | $\{a, b, c\}$ | $\{a, b, c\}$                                  | all derivatives non- $\Sigma^*$                                                          |
| $a \vee b$                               | $\{a, b, c\}$ | $\{a, b, c\}$                                  | $\partial_c(a \vee b) = \emptyset$ , still non- $\Sigma^*$                               |
| $a^*$                                    | $\{a, b\}$    | $\{a, b\}$                                     | $\partial_a(a^*) = a^*$ , non- $\Sigma^*$ ; $b$ starts $[b]$                             |
| $\neg a$ (double neg.)                   | $\{a, b\}$    | $[a]$                                          | $\partial_b(\neg a) = \Sigma^* \Rightarrow b$ excluded $^\dagger$                        |
| $a \cdot \Sigma^*$                       | $\{a, b\}$    | $[b]$                                          | $\partial_a(a \cdot \Sigma^*) = \Sigma^* \Rightarrow a$ excluded $^\dagger$              |
| $a \cdot \Sigma^* \vee b \cdot \Sigma^*$ | $\{a, b, c\}$ | $[c]$                                          | both $a, b$ excluded; only $c$ avoids $^\dagger$                                         |
| $\neg a \wedge \neg b$                   | $\{a, b, c\}$ | $\{a, b\}$                                     | De Morgan: $\neg(\neg a \wedge \neg b) = a \vee b$ ; $\partial_c$ is $\Sigma^{*\dagger}$ |

Table 2. LTL<sub>f</sub> to RE translation.  $\Sigma^1 \triangleq$  Single Wildcard,  $\neg\emptyset \triangleq \neg\emptyset$ .

| LTL formula        | RE $\llbracket \cdot \rrbracket$                                      |
|--------------------|-----------------------------------------------------------------------|
| $\top$             | $\neg\emptyset$                                                       |
| $\perp$            | $\emptyset$                                                           |
| $e$                | $\{e\}$                                                               |
| $\neg\phi$         | $\neg\llbracket\phi\rrbracket$                                        |
| $\phi \wedge \psi$ | $\llbracket\phi\rrbracket \cap \llbracket\psi\rrbracket$              |
| $\phi \vee \psi$   | $\llbracket\phi\rrbracket \cup \llbracket\psi\rrbracket$              |
| $X\phi$            | $\Sigma^1 \cdot \llbracket\phi\rrbracket$                             |
| $F\phi$            | $\neg\emptyset \cdot \llbracket\phi\rrbracket$                        |
| $G\phi$            | $\neg(\neg\emptyset \cdot \neg\llbracket\phi\rrbracket)$              |
| $\phi U \psi$      | $step(\phi)^* \cdot \llbracket\psi\rrbracket$ ( $\phi$ propositional) |

case advances both  $r_1$  and  $r_2$  in lockstep: for each event  $e$  that  $r_1$  can begin with, the residual after  $e$  is  $\partial_e(r_1) \setminus \partial_e(r_2)$ , consuming one event from each side simultaneously.

*Quick example.* Let the postcondition  $r_1 = \text{putMVar}(v)$  ( $\text{putMVar}(v)$  just occurred) and the obligation  $r_2 = \text{finally}(\text{putMVar}(v))$  ( $\text{putMVar}(v)$  must eventually occur). The event  $\text{putMVar}(v)$  in  $r_1$  satisfies  $\text{finally}(\text{putMVar}(v))$ , so the residual  $r_1 \setminus r_2 = \neg\emptyset$  and the obligation is fully discharged. Conversely, if  $r_1 = \text{send}(v)$  instead, then  $\text{send}(v) \setminus \text{finally}(\text{putMVar}(v)) = \text{finally}(\text{putMVar}(v))$ : a send event makes no progress toward the  $\text{putMVar}$  obligation, leaving it intact.

We write  $\setminus$  for subtraction uniformly throughout the paper and in all listings; the library itself exposes this as a two-argument top-level function rather than as an infix Haskell operator, so no clash with reserved syntax arises.

### 3.2 The Pledge Type

The future field is *data-dependent*: it is a function of the return value  $a$ , allowing obligations to refer to the specific resource handle that an operation produces. The helper `evalFuture` evaluates this function at the computation's own return value.

```
data Pledge eff a = Pledge
  { ret    :: a
  , pre    :: eff      -- what must have occurred before
  , post   :: eff      -- what this computation produces
  , future :: a → eff  -- obligation indexed by the return value
  }
```

```
evalFuture :: Pledge eff a → eff
evalFuture e = future e (ret e)
```

We call the value field `ret` rather than the more common `val` specifically to mirror `Monad`'s return and the mathematical presentation below; we use `ret` only as a record field name and never define a function of that name, so it does not collide with monadic return/pure.

### 3.3 Monad Instances

The pure function introduces a value with no effect and no obligation:

```
pure x = Pledge { ret = x, pre = universe, post = empty, future = \_ → universe }
```

The bind operator sequences two computations; write  $e_2 = f(\text{ret}(e))$ .

```
e >>= f = let fe = f (ret e) in Pledge
{ ret    = ret fe
, pre    = pre e ∧ (post e \ \ pre fe)
, post   = post e ◊ post fe
, future = \_ → (post fe \ \ future e (ret e)) ∧ future fe (ret fe)
}
```

*Future condition.* Each future function is applied to the corresponding `ret` before the obligation is propagated:

$$\text{future}(e \gg f) = (\text{post}(e_2) \setminus \text{future}(e)(\text{ret}(e))) \wedge \text{future}(e_2)(\text{ret}(e_2)) \quad (3)$$

This removes from  $e$ 's future obligation (evaluated at  $e$ 's return value) those traces that  $e_2$ 's postcondition satisfies, then intersects the residual with  $e_2$ 's own future obligation (evaluated at  $e_2$ 's return value). When the result normalises to  $\neg\emptyset$ , all obligations are discharged.

*Precondition.*

$$\text{pre}(e \gg f) = \text{pre}(e) \wedge (\text{post}(e) \setminus \text{pre}(e_2)) \quad (4)$$

The term  $\text{post}(e) \setminus \text{pre}(e_2)$  is the *residual precondition*: the part of  $e_2$ 's precondition not already discharged by  $e$ 's postcondition. Both must hold simultaneously in the same input history.

When  $\text{post}(e)$  does not cover  $\text{pre}(e_2)$ , the residual is  $\emptyset$ , flagging a violation. When it fully covers  $\text{pre}(e_2)$ , the residual is  $\varepsilon$  and the combined precondition reduces to  $\text{pre}(e)$  (the standard Hoare sequencing rule). We flag immediately, as a pointer forward to §9, that Equation (4) is exactly the place where `Pledge` departs from being a monad in the fully-generic, four-field sense: the `pre` field is carried alongside the monadic triple  $(\text{ret}, \text{post}, \text{future})$  as a Hoare-style contract annotation, and the three monad laws of §9 are stated for that triple only.

### 3.4 The Applicative Instance

The `Pledge` monad also satisfies the `Applicative` class, which provides `<*>` for combining a function-valued computation with an argument-valued computation *without* data-dependency between them. Writing  $e_f$  for the function computation and  $e_x$  for the argument computation:

```
ef <*> ex = Pledge
{ ret    = ret ef (ret ex)
, pre    = pre ef ∧ (post ef \ \ pre ex)
, post   = post ef ◊ post ex
, future = \_ → (post ex \ \ future ef (ret ef)) ∧ future ex (ret ex)
}
```

The structure mirrors  $\gg$ , i.e., preconditions and postconditions compose in the same way, and the future condition is the conjunction of the two evaluated futures after subtracting the continuation's postcondition from the first computation's future. The key difference is that `<*>` cannot make  $e_x$  *depend* on the return value of  $e_f$ ;  $\gg$  passes  $\text{ret}(e)$  into  $f$ , enabling the data-dependent future conditions exemplified by `tlsHandshake` and `takeMVar`. We define `<*>` independently, by direct analogy with  $\gg$ , rather than deriving it via `ap` from `Monad`; doing so lets the `Applicative` instance be given, and reasoned about, without committing to the full `Monad` machinery, which matters for the semiring-weighted instance of §6 where some obligation domains admit sequencing without needing data-dependent binding.

### 3.5 Effects: How the Specification Layer Meets Real Code

The presentation so far — and every Composable instance in this paper — treats `eff` as a pure data type: `RE`, `GuardedRE`, `WRE`, or `SL`. Correspondingly, every annotated primitive we define (`tlsHandshake`, `takeMVar`, `malloc`, `alloc`, ...) is a *pure* Pledge value: constructing it performs no IO, touches no `MVar`, and opens no file. This is deliberate, and it is worth being explicit about why, and about what it means for the claim that Pledge “integrates with the `mtl` stack.”

*Why `eff` is always pure.* A Pledge `eff` a value is a specification, not an implementation. Its three annotation fields describe a trace, a heap shape, or a weight — mathematical objects that subtraction can manipulate algebraically. If `eff` itself were IO or contained live resources, subtraction would need to run and compare real IO actions, which is neither decidable nor the point: the value of the derivative-based algebra of §2.2 is precisely that it operates on syntax, without executing anything. Keeping `eff` pure is what makes residual annotations computable at all.

*What “integrates with `mtl`” means concretely.* Pledge `eff` is an ordinary Monad instance, so it can be used with plain `do`-notation, sequence, and the other combinators any Haskell programmer already knows; no bespoke notation or `RebindableSyntax` is required, unlike the indexed-monad approaches discussed in §10. But we do *not* claim that a Pledge value replaces or transparently wraps an IO, `StateT`, or `ReaderT` computation, and no example in this paper runs Pledge and real effects in the same monad transformer stack. We revise the earlier, overstated phrasing of the abstract accordingly; what we mean is the narrower, and we believe still useful, claim that a Pledge specification can be constructed and composed using the same Monad/Applicative vocabulary that `mtl`-style code already uses, so a Haskell programmer does not need to learn a second composition discipline to read or write one.

*The intended integration pattern: a shadow specification.* Concretely, we envisage Pledge used as a *shadow* specification that runs alongside, but separately from, the real effectful code, checked once the specification term is fully built:

```
main = do
  let spec = tlsHandshake ctx >>= \c → tlsSend c >>= tlsByte c -- pure Pledge RE term
  assert (normalize (pre spec) == top) "precondition_violated"
  assert (normalize (evalFuture spec) == top) "future_obligation_unmet"
  realCtx <- Network.TLS.handshake realParams -- the actual IO effect
  ...
```

The `spec` value and the real IO computation are constructed from the same call sequence by convention, and their correspondence is the programmer’s responsibility, not something Pledge checks; keeping them in sync as the real code evolves is exactly the cost of this shadow-specification approach, and we discuss tighter integration (deriving the shadow automatically from an effect-handler row, so the two cannot drift) as future work in §11.

*On the type of future.* A closely related question is why `future` has type `a → eff` rather than simply `eff`, given that every example in §8 applies `future` only to `ret` (via `evalFuture`), never to an arbitrary argument. The answer is data-dependency at the point where `future` is *written*, not where it is applied: `takeMVar v` does not know its own return value when its Pledge record is constructed in isolation, so its future condition must be expressed as a function `\char`\\r → finally (Atom "putMVar" [Num r])` that names “whichever handle this operation returns.” The function type only becomes visible as a function inside `>>=`, where the record is applied to the *corresponding* `ret`, consistently, at both call sites in Equation (3). We chose to make `post` a plain `eff` rather than a function of the same shape because no example in this paper requires a postcondition to depend

on the value it is itself reporting; a symmetric design in which  $\text{post}$  is also  $a \rightarrow \text{eff}$ , applied to the previous computation's return value, is a reasonable and strictly more general alternative that we did not need for these case studies but do not rule out.

#### 4 The RE Instance

The Composable RE instance maps the algebra operations onto RE constructors and implements subtraction via derivatives:

```
instance Composable RE where
  concatenation = Seq
  conjunction  = And
  empty        = Epsilon
  universe     = anything      -- = Not Bot

  subtraction Epsilon r2 = r2
  subtraction (Not Bot) _ = Epsilon  -- Sigma* \ r = eps (trivially satisfied)
  subtraction r1 r2 =
    let alph = atoms r1 `union` atoms r2  -- combined alphabet
        evts = firstWith alph r1
        step e = subtraction (normalize (derivative e r1))
                              (normalize (derivative e r2))
    in foldr Or Bot (map step evts)

normalize :: RE → RE  -- RE-specific; not part of Composable
normalize r = {- ... algebraic simplification ... -}
```

The second clause handles the universal case:  $\neg \emptyset \setminus r = \varepsilon$  for any  $r$ . The general clause collects the alphabet from *both* operands before calling `firstWith`, ensuring events appearing only in  $r_2$  are still considered under complement via Equation (2). The `normalize` function is defined outside the instance and applies the RE-specific laws listed in Table 3, which exploit the complement constructor.

*Termination.* The recursive subtraction call terminates because `normalize` reduces every RE to a canonical representative and, for any regular expression  $r$ , the set of distinct normalised derivatives  $\{ [\partial_w(r)] \mid w \in \Sigma^* \}$  is *finite* — a classical result due to Brzozowski [Brzozowski 1964], refined to include complement by Owens et al. [Owens et al. 2009]. Each recursive call to subtraction is made with the normalised derivative of the *previous* operands, so the recursion explores a path in this finite set and must eventually reach a base case ( $\varepsilon$  or  $\neg \emptyset$ ) or a previously visited pair.

##### 4.1 Soundness

The examples of §8 read a fully-discharged residual ( $\neg \emptyset$  in future,  $\neg \emptyset$  in pre) as a certificate that the composed specification is contract-correct. We now state and prove the theorem that licenses this reading for the RE instance, addressing the soundness gap raised by both Reviewer A and the program committee.

For an RE value  $r$  and a finite trace  $w = a_1 a_2 \cdots a_n$  over events, write  $w \models r$  for  $w \in L(r)$ , the standard language membership relation, and  $\partial_w(r)$  for the iterated derivative  $\partial_{a_n}(\cdots \partial_{a_1}(r) \cdots)$ .

LEMMA 4.1 (DERIVATIVE CORRECTNESS). *For all  $r$  and all traces  $w$ ,  $w \models r$  if and only if  $v(\partial_w(r)) = \text{True}$ .*

Table 3. Normalisation laws applied by normalize. All rules are applied recursively bottom-up.

| Law                                                            | Constructor |
|----------------------------------------------------------------|-------------|
| $\varepsilon \cdot r = r, r \cdot \varepsilon = r$             | <b>Seq</b>  |
| $\emptyset \vee r = r, r \vee \neg\emptyset = \neg\emptyset$   | <b>Or</b>   |
| $\emptyset \wedge r = \emptyset, r \wedge \neg\emptyset = r$   | <b>And</b>  |
| $\neg\neg r = r$                                               | <b>Not</b>  |
| $\neg(r_1 \vee r_2) = \neg r_1 \wedge \neg r_2$                | <b>Not</b>  |
| $\neg(r_1 \wedge r_2) = \neg r_1 \vee \neg r_2$                | <b>Not</b>  |
| $\neg\emptyset = \neg\emptyset, \neg\neg\emptyset = \emptyset$ | <b>Not</b>  |
| $\emptyset^* = \varepsilon, \varepsilon^* = \varepsilon$       | <b>Star</b> |

PROOF. By induction on  $w$ . The base case  $w = \varepsilon$  is exactly the definition of  $v$ . For  $w = a \cdot w'$ ,  $w \models r$  iff  $w' \models \partial_a(r)$ , which is the standard derivative correctness property [Brzozowski 1964], extended to **Not** by Equation (1) and to **And** by the direct clause in §2.1; the inductive hypothesis on  $w'$  and  $\partial_a(r)$  then gives  $v(\partial_w(\partial_a(r))) = v(\partial_w(r))$ , as required.  $\square$

LEMMA 4.2 (SUBTRACTION CORRECTNESS). *For all  $r_1, r_2$  and all traces  $w$ : if  $w \models r_1$ , then  $w \models r_2$  if and only if  $\varepsilon \models \partial_w(r_1) \setminus \partial_w(r_2)$  is witnessed along the recursion, i.e.  $v(\partial_w(r_1) \setminus \partial_w(r_2))$  reduces, via the base cases of subtraction, to True exactly when  $v(\partial_w(r_2)) = \text{True}$ .*

PROOF. By induction on the structure of the subtraction recursion. The base case subtraction  $\varepsilon \ r_2 = r_2$  applies when  $r_1$ 's derivative has been driven to  $\varepsilon$ , i.e.  $w$  is exactly a word of  $L(r_1)$  with nothing left to consume; nullability of the residual is then nullability of  $r_2$  itself, matching Lemma 4.1. The base case subtraction  $(\neg\emptyset) \ r_2 = \varepsilon$  applies when  $r_1$ 's derivative has become the universal language, in which case every continuation is already in  $L(r_1)$ , so the residual should record “fully observed,” i.e. nullable unconditionally, which  $\varepsilon$  is. The recursive case advances  $r_1$  and  $r_2$  by the same event  $e$  and applies the inductive hypothesis at  $\partial_e(r_1), \partial_e(r_2)$ .  $\square$

THEOREM 4.3 (SOUNDNESS OF RE-RESIDUAL CHECKING). *Let  $e :: \text{Pledge RE}$  be a specification term built from pure and  $\gg$  applications of RE-annotated primitives, and let  $w$  be any finite trace of events actually emitted by a terminating execution whose call structure matches  $e$  (i.e.  $w$  is a concatenation of the post events of the primitives invoked, in invocation order). If  $\text{normalize}(\text{evalFuture } e) = \neg\emptyset$  and  $\text{normalize}(\text{pre } e) = \neg\emptyset$ , then  $w$  satisfies every future condition and every precondition named by the primitives composing  $e$ .*

PROOF SKETCH. By induction on the structure of  $e$ . For  $e = \text{pure } x$ , both residuals are  $\neg\emptyset$  by definition and there are no obligations to satisfy, so the claim holds vacuously. For  $e = e_1 \gg f$ , the residual future of  $e$  is given by Equation (3), which by Lemma 4.2 is  $\neg\emptyset$  exactly when the trace segment corresponding to  $f(\text{ret}(e_1))$ 's postcondition discharges  $e_1$ 's future obligation (evaluated at  $\text{ret}(e_1)$ ) and  $f(\text{ret}(e_1))$ 's own future obligation is separately  $\neg\emptyset$ ; the inductive hypothesis applied to  $e_1$  and to  $f(\text{ret}(e_1))$  then gives that the corresponding trace segments satisfy their respective obligations, and concatenating these segments (by the definition of  $w$ ) gives that  $w$  satisfies the future obligation of  $e$ . The precondition case is symmetric, using Equation (4) and Lemma 4.2 in place of Equation (3).  $\square$

Theorem 4.3 is stated for the RE instance because its proof leans on the derivative correctness of Lemma 4.1, which is specific to regular languages. The GRE instance inherits this soundness for its RE component and additionally requires the Presburger side to be checked by a sound

decision procedure (Z3), which we take as given rather than re-proving. The WRE instance admits an analogous statement with  $v$  replaced by  $w\text{Nullable}$  and the conclusion weakened to a numeric bound on the accumulated weight, which we state but do not prove in detail. The SL instance, as discussed in §7, does not admit an analogous soundness theorem as it stands, precisely because its subtraction records the wand symbolically without a semantic model of the heap; establishing one is exactly the “real solver” extension identified as future work in §11.

## 5 The GuardedRE Instance

The RE instance reasons about event traces but is entirely silent about arithmetic state. The GRE instance closes this gap: a guarded regular expression pairs a trace RE with a Presburger arithmetic predicate over a concrete heap, so that a single annotation can simultaneously enforce a protocol ordering *and* an arithmetic bound.

### 5.1 The GuardedRE Type

```
data GuardedRE = GuardedRE PPred RE
    deriving (Eq)
```

A state (heap, trace) satisfies **GuardedRE**  $p$   $r$  iff

$$\text{heap} \models p \quad \wedge \quad \text{trace} \in L(r).$$

The two sides are independent:  $p$  is a static constraint on the heap, while  $r$  advances through Brzozowski derivatives as events arrive.

Two smart constructors lift a pure RE or a pure Presburger predicate:

```
fromRE    :: RE      → GuardedRE
fromRE r   = GuardedRE PTrue r      -- no heap constraint

fromPPred :: PPred → GuardedRE
fromPPred p = GuardedRE p universe -- accept any trace
```

### 5.2 The Composable GuardedRE Instance

```
instance Composable GuardedRE where
    concatenation (GuardedRE p1 r1) (GuardedRE p2 r2) =
        GuardedRE (normalizePPred (PAnd p1 p2)) (normalize (Seq r1 r2))
    conjunction (GuardedRE p1 r1) (GuardedRE p2 r2) =
        GuardedRE (normalizePPred (PAnd p1 p2)) (normalize (And r1 r2))
    empty       = GuardedRE PTrue Epsilon
    universe    = GuardedRE PTrue (Not Bot) -- = anything
    subtraction (GuardedRE p1 r1) (GuardedRE p2 r2) =
        GuardedRE (normalizePPred (PAnd p1 p2))
            (normalize (reSubtraction r1 r2))
```

Both concatenation and subtraction conjoin both heap predicates: the residual must respect the constraints from both operands. The RE component uses the same `reSubtraction` and `normalize` as the plain RE instance (named `reSubtraction` here purely to distinguish it, in this listing, from the outer **GuardedRE** subtraction being defined; it is the identical function called `subtraction` in §4).

### 5.3 Presburger Expressions and Normalisation

Presburger predicates are built over a small arithmetic expression language:

```
data PExpr = Lit Int | ValAt Addr | PAdd PExpr PExpr | PSub PExpr PExpr

data PPred
  = PTrue | PFalse      -- canonical true/false
  | PLt PExpr PExpr | PLe PExpr PExpr | PEq PExpr PExpr | PGe PExpr PExpr
  | PNot PPred | PAnd PPred PPred
```

`Lit k` is the integer literal  $k$ ; `ValAt a` reads the current value stored at heap address  $a$  (so, e.g., `PLt (ValAt a) (Lit maxVal)` is the predicate  $h[a] < \text{maxVal}$  used in §8.5); `PAdd/PSub` build compound arithmetic expressions from these.

Composing multiple `GuardedRE` values accumulates a tree of `PAnd` nodes. The `normalizePPred` function flattens this tree, removes duplicates, substitutes equalities (e.g.  $h[a] = k \Rightarrow$  replace `ValAt a` by `Lit k` throughout), evaluates literal comparisons, and replaces  $\neg \text{true}$  with the canonical false constructor `PFalse`:

```
normalizePPred :: PPred → PPred
normalizePPred p = rebuildAnd (substEqs eqs atoms)
  where
    atoms = flattenAnd (normStep p)
    eqs    = [ (a, k) | PEq (ValAt a) (Lit k) <- atoms ]
```

The introduction of `PFalse` prevents the solver from being invoked with expressions such as  $(\neg \text{true}) \wedge \dots$  that are syntactically unsatisfiable; these normalise immediately to `PFalse` without any solver call. Predicates that survive this purely syntactic normalisation step are discharged by an SMT call to Z3 (via SBV) to determine satisfiability.

### 5.4 Example: Bounded Counter

A bounded counter stored at heap address  $a$  with maximum value  $M$  must follow the protocol `init · (inc | dec)* · snapshot`. Increment is additionally guarded by  $h[a] < M$  (no overflow); decrement by  $h[a] > 0$  (no underflow).

```
initCounter :: Addr → Pledge GuardedRE ()
initCounter a = Pledge
  { ret = ()
  , pre = fromRE universe
  , post = GuardedRE (PEq (ValAt a) (Lit 0))
                (Single (Atom "init" [Num a]))
  , future = \_ → fromRE universe }

increment :: Addr → Int → Pledge GuardedRE ()
increment a maxVal = Pledge
  { ret = ()
  , pre = GuardedRE (PLt (ValAt a) (Lit maxVal)) universe
  , post = fromRE (Single (Atom "inc" [Num a]))
  , future = \_ → fromPPred (PGe (ValAt a) (Lit 0)) }
```

The pre of increment carries  $h[a] < M$  (pure Presburger) conjoined with the universal RE. After composing eleven increments against a bound of 10, normalizePPred derives the contradiction  $h[a] < 10 \wedge \neg(h[a] < 10) \equiv \text{false}$ , flagging the overflow before any execution takes place.

*Bad programs and residuals.*

| Bad program                | Residual pre (Presburger) | Residual pre (RE) |
|----------------------------|---------------------------|-------------------|
| overflow (11 incs, max=10) | <b>false</b>              | $\neg\emptyset$   |
| underflow (dec at zero)    | <b>false</b>              | $\neg\emptyset$   |
| noInit (skip init)         | $\neg\emptyset$           | $\emptyset$       |

## 6 The WeightedRE Instance

The RE and GRE instances are Boolean: a future condition is either satisfied or violated. The WRE instance generalises membership to a *weight* drawn from a semiring, enabling probabilistic and quantitative obligations to be expressed within the same Composable framework.

### 6.1 The Semiring Class

```
class (Eq w, Show w) => Semiring w where
  szero :: w          -- additive identity (blocked path)
  sone  :: w          -- multiplicative identity
  sadd  :: w -> w -> w
  smul  :: w -> w -> w
```

Setting  $w = \text{Bool}$  (with  $\text{sadd} = (\vee)$ ,  $\text{smul} = (\wedge)$ ) recovers the Boolean RE instance. Two further instances capture quantitative scenarios:

| Instance                   | szero     | sadd                 | smul             |
|----------------------------|-----------|----------------------|------------------|
| <b>Prob</b> (probability)  | 0         | $\min(p_1 + p_2, 1)$ | $p_1 \times p_2$ |
| <b>Tropical</b> (min-cost) | $+\infty$ | min                  | +                |

In the probability semiring,  $\text{sadd}$  is bounded addition (sum of probabilities capped at 1) and  $\text{smul}$  is multiplication. In the tropical semiring,  $\text{sadd}$  is minimum (taking the cheaper alternative) and  $\text{smul}$  is addition (summing costs along a path).

### 6.2 The WRE Type and Operators

```
data WRE w
  = WBot          -- weight szero: no accepting path
  | WEps w        -- weight w: accept the empty trace
  | WSingle w Event -- weight w: accept exactly this event
  | WSeq (WRE w) (WRE w)
  | WAdd (WRE w) (WRE w)
  | WAnd (WRE w) (WRE w)
  | WStar (WRE w)
```

The generalised nullable function accumulates the weight of all accepting paths through the empty trace:

```
wNullable :: Semiring w => WRE w -> w
wNullable (WEps w)      = w
wNullable (WSeq r1 r2) = smul (wNullable r1) (wNullable r2)
```

```

wNullable (WAdd r1 r2) = sadd (wNullable r1) (wNullable r2)
wNullable (WAnd r1 r2) = smul (wNullable r1) (wNullable r2)
wNullable (WStar r)    = sone
wNullable _           = szero

```

The functions `wDerivative` and `wSubtraction` follow the same structure as their Boolean counterparts, with semiring operations substituted for Boolean ones. The `Composable (WRE w)` instance (enabled by `FlexibleInstances`) maps concatenation to `WSeq`, conjunction to `WAnd`, and subtraction to `wSubtraction`.

### 6.3 Derived Forms

```

wTop      :: Semiring w => WRE w
wTop      = WStar (WSingle sone Wildcard)  -- sone-weighted anything

wFinally :: Semiring w => w -> Event -> WRE w
wFinally weight e = WSeq wTop (WSingle weight e)

wGlobally :: Semiring w => w -> Event -> WRE w
wGlobally weight e = WStar (WSingle weight e)

```

### 6.4 Example: Probabilistic Memory (Prob Semiring)

In a probabilistic setting, each `free(a)` has an associated probability of being called. Each `malloc` call carries a future condition asserting that `free(a)` must be called with probability at least  $p$ :

```

malloc :: Addr -> Double -> Pledge (WRE Prob) Addr
malloc a p = Pledge
  { ret = a, pre = wTop
  , post = WSingle sone (Atom "malloc" [Num a])
  , future = \r -> wFinally (Prob p) (Atom "free" [Num r]) }

```

The residual weight of `wNullable` at the end of a program measures how much of the obligation has been satisfied: a weight of `Prob 1.0` indicates that the obligation is fully discharged; a weight strictly less than 1 quantifies the probability shortfall. A call `malloc a 0.9` whose corresponding `free a` is never composed into the program leaves a residual weight of exactly `Prob 0.9`: the obligation is not a contract the caller must write, since the caller cannot control the run’s probabilistic outcome; it is the declared quantitative claim itself, and the residual reports how much of that claim the composed call sequence still owes, which is the reading we intend when we describe WRE obligations as “an output of an analysis” in §1.

### 6.5 Example: Min-Cost Scheduling (Tropical Semiring)

Under the tropical semiring, `WSingle (Tropical c) e` assigns cost  $c$  to emitting event  $e$ ; `wNullable` then returns the minimum total cost of reaching an accepting state:

```

submitJob :: Int -> Double -> Pledge (WRE Tropical) ()
submitJob jobId cost = Pledge
  { ret = ()
  , pre = wTop
  , post = WSingle (Tropical cost) (Atom "submit" [Num jobId])
  , future = \_ ->
    wFinally (Tropical 0) (Atom "complete" [Num jobId]) }

```

Composing several `submitJob` calls yields a future whose `wNullable` is the minimum total cost of completing all submitted jobs. A program that omits a complete event for some job  $i$  retains `wFinally (Tropical 0) (complete i)` in the residual, and the overall weight is  $+\infty$  (i.e. `szero`), flagging the missing completion.

*Absence of weighted complement.* The `WRE` type does not include a `WNot` constructor. This is a deliberate restriction: complement over a semiring weight does not have a canonical definition, because Boolean negation ( $0 \leftrightarrow 1$ ) does not generalise uniformly to an arbitrary semiring (e.g. what is the “negation” of a probability 0.7, or of a tropical cost 5?). As a consequence, the  $\text{LTL}_f$  embedding from Section 2.3 does not transfer to the `WRE` instance: operators that rely on complement — `G`, `U`, and `never` — are unavailable. The operators `wFinally` and `wGlobally` above are defined directly via `WStar` and `WSeq` without complement, and cover the common quantitative patterns. This is the concrete sense, referenced in §1, in which `WRE` is fully usable for the obligations it targets while giving up part of the `RE` instance’s generality.

## 7 The SL Instance

The `Composable` typeclass is not inherently tied to trace-based specifications. We demonstrate its generality with a fourth instance over *separation-logic heap predicates*, where future conditions describe what heap state must hold upon completion of the computation. As stated in §1, this instance is included primarily to show that `Composable` generalises beyond traces; we are explicit throughout this section, and again in §4.1, about the respects in which it falls short of the `RE` instance’s guarantees.

### 7.1 Structural Parallel with RE

Table 4 shows the role each SL construct plays in the `Composable` interface.

Table 4. Structural parallel between the four `Composable` instances.

| Operation     | RE                  | GRE                                | $\text{WRE}_w$                      | SL              |
|---------------|---------------------|------------------------------------|-------------------------------------|-----------------|
| concatenation | $r_1 \cdot r_2$     | $(p_1 \wedge p_2, r_1 \cdot r_2)$  | <code>WSeq</code>                   | $P * Q$         |
| conjunction   | $r_1 \wedge r_2$    | $(p_1 \wedge p_2, r_1 \wedge r_2)$ | <code>WAnd</code>                   | $P \wedge Q$    |
| empty         | $\varepsilon$       | $(\text{true}, \varepsilon)$       | <code>WEps</code> <code>sone</code> | <b>emp</b>      |
| universe      | $\neg\emptyset$     | $(\text{true}, \neg\emptyset)$     | <code>wTop</code>                   | $\top$          |
| subtraction   | Brzozowski quotient | quotient (RE) + $\wedge$ (PPred)   | <code>wSubtraction</code>           | $P \multimap Q$ |

The `bind` formula is unchanged:  $\text{pre}(e \gg f) = \text{pre}(e) \wedge (\text{post}(e) \multimap \text{pre}(e_2))$  and  $\text{future}(e \gg f) = (\text{post}(e_2) \multimap \text{future}(e)) \wedge \text{future}(e_2)$ . Here  $\multimap$  is the magic wand (separating implication):  $P \multimap Q$  holds of heap  $h$  iff for every heap  $h'$  disjoint from  $h$  satisfying  $P$ , the combined heap  $h \cup h'$  satisfies  $Q$ .

### 7.2 The SL Predicate Type

```
data SL
= Emp          -- empty heap          (identity for SepStar)
| Top          -- any heap            (identity for Conj)
| Pure PPred   -- pure Presburger constraint (heap-independent)
| Cell Addr Val -- singleton: address a holds value v
| SepStar SL SL -- P * Q separating conjunction
| Conj SL SL   -- P ∧ Q ordinary conjunction
```

```
| Wand SL SL      -- P -* Q magic wand (residual)
deriving (Eq, Show)
```

### 7.3 Separating vs. Ordinary Conjunction

SepStar  $P \ Q$  holds of heap  $h$  iff  $h$  splits into disjoint parts  $h_1 \uplus h_2$  with  $P(h_1)$  and  $Q(h_2)$ ; it is the standard separating conjunction of ownership. Conj  $P \ Q$  holds of  $h$  iff the *same* heap satisfies both  $P$  and  $Q$ . Without pure constraints, Conj on purely spatial predicates is either trivial ( $\top \wedge P = P$ ) or unsatisfiable ( $\text{Cell } l_1 \ v_1 \wedge \text{Cell } l_2 \ v_2 = \text{false}$  for  $l_1 \neq l_2$ ). The constructor becomes useful when one side is a pure constraint  $[\phi] \wedge (\ell \mapsto v)$ , combining a heap-independent condition with spatial ownership.

### 7.4 Presburger Arithmetic for Pure Constraints

Pure constraints reuse the same PPred/PEXpr language introduced in Section 5 (normalizePPred, and the reading of ValAt  $a$  as the value currently stored at address  $a$ ). The predicate Pure  $p$  holds of any heap for which evalPPred  $h \ p$  is true; it enforces no ownership of cells. The smart constructor pureSL realises the standard SL box notation: pureSL **PTrue** = Emp and pureSL  $p = \text{Conj } (\text{Pure } p) \text{ Emp}$  for non-trivial  $p$ .

### 7.5 The Composable SL Instance

```
instance Composable SL where
  concatenation = SepStar
  conjunction   = Conj
  empty         = Emp
  universe      = Top

  subtraction Emp q = q      -- emp -* Q = Q
  subtraction Top _ = Emp    -- Top trivially covers any pre
  subtraction p  q = Wand p q -- general: symbolic wand
```

The two base cases mirror those of the RE instance:  $\varepsilon \setminus r = r$  becomes **emp**  $\neg * Q = Q$  (providing no heap leaves the obligation unchanged), and  $\neg \emptyset \setminus r = \varepsilon$  becomes  $\top \neg * Q = \text{emp}$  (a  $\top$  postcondition covers any precondition, leaving no residual). The general case records the wand symbolically for later evaluation or normalisation.

### 7.6 Normalisation

The normalizeSL function applies the standard SL algebraic identities:

| Law                                         | Constructor |
|---------------------------------------------|-------------|
| <b>emp</b> $* Q = Q$ , $P * \text{emp} = P$ | SepStar     |
| $\top \wedge Q = Q$ , $P \wedge \top = P$   | Conj        |
| $P \wedge P = P$ (idempotent)               | Conj        |
| <b>emp</b> $\neg * Q = Q$                   | Wand        |
| $P \neg * \top = \top$                      | Wand        |

*Symbolic recording, not semantic discharge.* It is important to distinguish what the SL instance does from what it does not do. The five normalisation rules above reduce *syntactically* trivial wands (e.g. **emp**  $\neg * Q = Q$ ), but the general Wand  $p \ q$  case is recorded *symbolically*: no heap model is consulted, and no solver is invoked. A residual such as Wand (Cell 0 42) (Cell 0 99) is retained as a data

structure even though it is semantically unsatisfiable ( $0 \mapsto 42$  and  $0 \mapsto 99$  cannot simultaneously hold). The SL instance therefore *records* obligations and propagates them correctly through bind; it does not *check* whether the accumulated residual is satisfiable or entailed, and consequently Theorem 4.3’s style of guarantee does not extend to it (§4.1). Connecting residual SL predicates to an external solver (e.g. Z3 via the sbv library, or a dedicated SL decision procedure such as Smallfoot) is identified in Section 11 as the natural next engineering step. All claims of “violation detection” for the SL instance in the examples below should be read as “the residual annotation is non-trivially non-universe”, which a programmer or tool can then inspect further, exactly the caveat flagged for the SL row of Table 4.

## 8 Examples

We demonstrate Pledge across use cases spanning all four instances. Each example defines primitive operations annotated with pre, post, and future, composes them in the Pledge monad, and inspects the normalised annotations of the result. A future of  $\neg\emptyset$  (or `wNullable = some` for WRE) and a pre of  $\neg\emptyset$  together certify temporal correctness (Theorem 4.3, for RE). Where an obligation domain admits one, we favour examples whose future condition is genuinely non-trivial, since a universe future condition throughout a derivation would make no use of the propagation machinery this paper is about.

### 8.1 TLS Handshake Lifecycle

The `tls` library obligation is representative of a broad class of Haskell protocol obligations: a successful handshake must always be paired with a subsequent bye. We model the Context as an integer identifier.

```
type Context = Int
```

```
tlsHandshake :: Context → Pledge RE Context
tlsHandshake ctx = Pledge
  { ret    = ctx
  , pre    = universe
  , post   = Single (Atom "handshake" [Num ctx])
  , future = \c → finally (Atom "bye" [Num c]) }
```

```
tlsBye :: Context → Pledge RE ()
tlsBye ctx = Pledge
  { ret    = ()
  , pre    = Single (Atom "handshake" [Num ctx])
  , post   = Single (Atom "bye" [Num ctx])
  , future = \_ → universe }
```

```
tlsSend :: Context → Pledge RE ()
tlsSend ctx = Pledge
  { ret    = ()
  , pre    = Single (Atom "handshake" [Num ctx])
  , post   = Single (Atom "send" [Num ctx])
  , future = \_ → universe }
```

```
-- Good: handshake followed by data exchange then bye
goodSession = do
```

```

ctx <- tlsHandshake 1
tlsSend ctx
tlsBye ctx

-- Bad future: bye omitted; residual = finally(bye(1))
missingBye = do
  ctx <- tlsHandshake 1
  tlsSend ctx

-- Bad future: two contexts opened, only one closed
partialClose = do
  ctx1 <- tlsHandshake 1
  ctx2 <- tlsHandshake 2
  tlsBye ctx1

```

The program `missingBye` retains `finally(bye(1))` as its residual future, naming the unclosed context precisely. Meanwhile, `partialClose` retains `finally(bye(2))`: the future condition is data-dependent, so the residual identifies context 2 specifically, not a generic “some context was not closed.”

## 8.2 MVar Lifecycle

The `takeMVar` operation returns the taken handle; its data-dependent future condition uses the lambda `r` so the obligation tracks the *returned* handle. The `putMVar` operation carries as its future obligation `noUntil(putMVar(v), takeMVar(v))`, forbidding a second `putMVar(v)` until `takeMVar(v)` resets the guard. (The full definitions of `takeMVar` and `putMVar` appear in Section 3.2.) As discussed in §3.5, `MVar` here is the pure type synonym `type MVar = Int` used by the specification, standing in for a real `Control.Concurrent.MVar` handle; see §11 for how a real program’s `MVar` usage would be linked to this specification.

```

-- Good: data-dependent - obligation discharged on the returned handle
takeAndPut n = do { v <- takeMVar n; putMVar v }

-- Good: re-acquire same MVar after releasing - noUntil is reset by takeMVar
takePutTakePut = do { v <- takeMVar 1; putMVar v; v <- takeMVar 1; putMVar v }

-- Bad future: putMVar(2) obligation remains (MVar 2 never returned)
missingPut = do { v1 <- takeMVar 1; putMVar v1; _ <- takeMVar 2; return () }

-- Bad pre: takeMVar(1) precondition not met
putWithoutTake = putMVar 1

-- Bad future: double-put immediately collapses future to Bot
doublePutImmediate = do { v <- takeMVar 1; putMVar v; putMVar v }

-- Bad future: double-put with intervening work, still collapses to Bot
doublePutWithWork = do
  { v1 <- takeMVar 1; v2 <- takeMVar 2; putMVar v1; putMVar v2; putMVar v1 }

```

The three examples below return non-unit values, showing that future conditions are tracked regardless of the return type.

```
-- Good: ret = MVar; future = finally(putMVar(1)) - obligation visible in residual
takeReturnHandle :: Pledge RE MVar
takeReturnHandle = takeMVar 1

-- Bad future: ret = (MVar,MVar); future = finally(putMVar(1)) ^ finally(putMVar(2))
takeTwoReturnPair :: Pledge RE (MVar, MVar)
takeTwoReturnPair = do { v1 <- takeMVar 1; v2 <- takeMVar 2; return (v1, v2) }

-- Good: ret = MVar; future = noUntil(putMVar(1),takeMVar(1)) - no pending finally
takePutReturnHandle :: Pledge RE MVar
takePutReturnHandle = do { v <- takeMVar 1; putMVar v; return v }
```

### 8.3 Mutex / Lock Lifecycle

```
release :: Int → Pledge RE ()
release m = Pledge
  { ret = ()
  , pre  = Single (Atom "acquire" [Num m])
  , post = Single (Atom "release" [Num m])
  , future = universe }

-- Bad future: lock 1 not released
lockLeak = do { acquire 1; acquire 2; release 2 }

-- Bad pre: release without acquire
releaseWithoutAcquire = release 1
```

### 8.4 Database Transactions

The `beginTx` operation carries a *disjunctive* future condition requiring that the transaction is eventually either committed or rolled back; `commit` and `rollback` each discharge this obligation:

```
beginTx :: Pledge RE ()
beginTx = Pledge
  { ret = (), pre = universe
  , post  = Single (Atom "beginTx" [])
  , future = Or (finally (Atom "commit" []))
                (finally (Atom "rollback" [])) }

commit :: Pledge RE ()
commit = Pledge
  { ret = (), pre = Single (Atom "beginTx" [])
  , post  = Single (Atom "commit" [])
  , future = universe }

rollback :: Pledge RE ()
rollback = Pledge
  { ret = (), pre = Single (Atom "beginTx" [])
  , post  = Single (Atom "rollback" []) }
```

```

    , future = universe }

-- Bad future: transaction never closed
openTx = do { beginTx }

-- Good: transaction properly committed
committedTx = do { beginTx; commit }

-- Good: transaction properly rolled back
abortedTx = do { beginTx; rollback }

```

openTx retains the full disjunction  $\text{finally}(\text{commit}) \vee \text{finally}(\text{rollback})$  as its residual future. committedTx and abortedTx fully discharge the obligation, yielding  $\neg\emptyset$ .

*RE summary.* Table 5 summarises the three RE domains.

Table 5. RE instance: bad programs and residual annotations.

| Domain      | Bad program             | Residual future                                                        | Residual pre    |
|-------------|-------------------------|------------------------------------------------------------------------|-----------------|
| Memory      | missingFree             | $\text{finally}(\text{free}(2))$                                       | $\neg\emptyset$ |
| Memory      | freeWithoutMalloc       | $\neg\emptyset$                                                        | $\emptyset$     |
| Memory      | doubleFreeImmediate     | $\emptyset$                                                            | $\neg\emptyset$ |
| Memory      | leakAndDoubleFree       | $\emptyset$                                                            | $\neg\emptyset$ |
| Memory      | allocReturnAddr         | $\text{finally}(\text{free}(1))$                                       | $\neg\emptyset$ |
| Memory      | allocTwoReturnPair      | $\text{finally}(\text{free}(1)) \wedge \text{finally}(\text{free}(2))$ | $\neg\emptyset$ |
| Mutex       | acquire 1 leaked        | $\text{finally}(\text{release}(1))$                                    | $\neg\emptyset$ |
| Mutex       | release without acquire | $\neg\emptyset$                                                        | $\emptyset$     |
| Transaction | no commit/rollback      | $\text{finally}(\text{commit}) \vee \text{finally}(\text{rollback})$   | $\neg\emptyset$ |

### 8.5 Bounded Counter (GRE Instance)

The bounded counter (§5) demonstrates that both the Presburger and RE components are necessary. The RE alone would accept overflow sequences; the Presburger predicate alone would accept arbitrarily ordered events.

```

-- Good: init, two increments, one decrement, snapshot
normalUse :: Pledge GuardedRE ()
normalUse = do
  initCounter 0; increment 0 10; increment 0 10
  decrement 0; snapshot 0

-- Bad: 11 increments against max 10
overflow :: Pledge GuardedRE ()
overflow = do
  initCounter 0
  sequence_ (replicate 11 (increment 0 10))
  snapshot 0

```

For overflow, the combined pre after composing all eleven increments normalises to  $[\text{false}] \wedge \neg\emptyset$ : the Presburger side collapses to **PFalse** while the RE side remains the universal language, pinpointing the arithmetic violation.

GRE *summary*.

| Bad program | Residual pre (GRE)              | Cause                |
|-------------|---------------------------------|----------------------|
| overflow    | <b>[false]</b> $\neg \emptyset$ | $h[0] < 10$ violated |
| underflow   | <b>[false]</b> $\neg \emptyset$ | $h[0] > 0$ violated  |
| noInit      | <b>[true]</b> $\emptyset$       | init event missing   |

## 8.6 Task Scheduling (WRE Instance)

The task-scheduling example shows that the tropical semiring naturally accumulates minimum costs. Each `submitJob` operation carries both a submission cost and a future condition requiring that every submitted job eventually completes.

```
-- Good: submit two jobs, complete both
goodSchedule :: Pledge (WRE Tropical) ()
goodSchedule = do
  submitJob 1 2.0; submitJob 2 3.0
  completeJob 1;  completeJob 2

-- Bad: job 2 never completed
missingComplete :: Pledge (WRE Tropical) ()
missingComplete = do
  submitJob 1 2.0; submitJob 2 3.0
  completeJob 1
```

For `goodSchedule`, `wNullable (evalFuture ...)` returns **Tropical**  $\emptyset$ , indicating all obligations are discharged at zero residual cost. For `missingComplete`, the residual future still contains `wFinally (Tropical  $\emptyset$ ) (complete 2)`, and the overall weight is  $+\infty$ , flagging job 2 as unfinished.

The probability-semiring memory example follows analogously: a call `malloc a p` with  $p = 0.9$  leaves a residual weight of 0.9 if `free a` is never called, dropping to 0 when it is, matching the reading given in §6.

## 8.7 Heap Memory (SL Instance)

The heap-memory example mirrors the RE memory example but uses heap ownership rather than event traces. Unlike the earlier definitions in this paper, `alloc` and `free` here use `Top` futures because, in this particular example, no cell's disposal is a precondition for allocating another; the obligation of interest is entirely in post (what is currently owned), which is what the leak example below exercises.

```
alloc :: Addr → Val → Pledge SL ()
alloc a v = Pledge
  { ret = (), pre = Top, post = Cell a v, future = Top }

free :: Addr → Val → Pledge SL ()
free a v = Pledge
  { ret = (), pre = Cell a v, post = Emp, future = Top }

writeCell :: Addr → Val → Val → Pledge SL ()
writeCell a old new = Pledge
  { ret = (), pre = Cell a old, post = Cell a new, future = Top }
```

**Good:** `alloc 0 1 >> alloc 1 2`. The combined post is SepStar (Cell 0 1) (Cell 1 2): two disjoint ownerships composed by separating conjunction, correctly reflecting that the two cells reside at distinct addresses.

**Bad:** `leak (alloc 0 1 >> alloc 1 2 >> free 0 1)`. After freeing only address 0, the post retains Cell 1 2, naming the unreleased ownership at address 1. To be precise about what “bad” means for this particular program: the residual post of Cell 1 2 is not itself a defect (it simply and correctly records that address 1 is still owned); the point of the example is diagnostic, showing that Pledge’s residual, after composing exactly these three operations, names address 1 and not address 0 as the one still outstanding, which is the piece of information a leak-detection tool built on top of this instance would want. This is different in kind from the RE memory examples of Table 5, where a non-universe residual is unambiguously a violation of a stated future obligation; here, since `alloc`’s future is Top, there is no such stated obligation, and it is the *caller*, inspecting post, who decides what counts as a leak.

**Bad:** `read without ownership (readCell 0 42)`. The pre of the bare read is Cell 0 42, which is never discharged, flagging the missing ownership.

## 8.8 Bank Account with Presburger Guards (SL Instance)

The bank account example demonstrates Pure Presburger constraints inside Conj, and, unlike the heap-memory example above, gives `withdraw` a non-trivial future condition: once money has been withdrawn against an account, the account must eventually be reconciled by a `closeAccount` or a further balancing deposit, so that a withdrawal cannot simply be forgotten about.

```
withdraw :: Addr → Val → Val → Pledge SL ()
withdraw a old amount = Pledge
  { ret = ()
  , pre = Conj (Pure (PGe (ValAt a) (Lit amount))) (Cell a old)
  , post = Cell a (old - amount)
  , future = \_ → SepStar (Wand (Cell a (old - amount)) Top)
                                (Pure PTrue) } -- must eventually reconcile a's cell

closeAccount :: Addr → Val → Pledge SL ()
closeAccount a bal = Pledge
  { ret = ()
  , pre = Conj (Pure (PEq (ValAt a) (Lit 0))) (Cell a bal)
  , post = Emp, future = Top }
```

The precondition of `withdraw` is a Conj of a pure Presburger guard ( $h[a] \geq \text{amount}$ ) and a spatial ownership claim (Cell  $a$  old). Without the Pure constructor, Conj of two purely spatial predicates would be either trivial or unsatisfiable (see §7); the combination of Pure and Conj is what makes value-dependent preconditions expressible.

**Bad: overdraft.** After `deposit 0 0 100`, the account holds 100. The call `withdraw 0 100 150` carries  $\text{pre} = \text{Conj} (\text{Pure} (h[0] \geq 150)) (\text{Cell } 0 \ 100)$ . Since  $100 < 150$ , the pure guard is not satisfiable, and the residual pre of the composed program retains this unsatisfiable constraint.

**Bad: unreconciled withdrawal.** `deposit 0 0 100 >> withdraw 0 100 40` leaves the residual future as SepStar (Wand (Cell 0 60)  $\top$ ) (Pure true), recorded symbolically per §7: composing a subsequent `closeAccount 0 60` discharges it, matching the RE instance’s disjunctive-obligation pattern from the transaction example (§8), but here over heap ownership instead of a trace.

### 8.9 Linked List (SL Instance)

A singly-linked-list node occupies two consecutive cells: address  $a$  holds the payload value and address  $a+1$  holds the next pointer.

```
allocNode :: Addr → Val → Val → Pledge SL ()
allocNode a val next = Pledge
  { ret = (), pre = Top
  , post = SepStar (Cell a val) (Cell (a+1) next)
  , future = Top }
```

**Two-node list** (`allocNode 0 10 2 >> allocNode 2 20 (-1)`). The combined post is a four-cell SepStar:

$$(\text{Cell } 0 \ 10 * \text{Cell } 1 \ 2) * (\text{Cell } 2 \ 20 * \text{Cell } 3 \ -1)$$

This is the canonical representation of a two-node list in separation logic: all four cells are disjointly owned, and the structure of SepStar mirrors the structure of the list.

*SL summary.* Table 6 summarises the four SL domains. The key difference from the RE instance is that residual annotations are heap predicates rather than traces: a leaked cell appears in post, an unsatisfiable ownership constraint in pre, and, for the bank account, an outstanding wand in future.

Table 6. SL instance: bad programs and residual annotations.

| Domain       | Bad program               | Residual post    | Residual pre        | Residual future         |
|--------------|---------------------------|------------------|---------------------|-------------------------|
| Heap memory  | Cell 1 2 not freed        | Cell 1 2 (owned) | $\top$              | $\top$                  |
| Heap memory  | read without ownership    | <b>emp</b>       | Cell 0 42           | $\top$                  |
| Bank account | insufficient balance      | Cell 0 100       | unsatisfiable guard | $\top$                  |
| Bank account | withdrawal not reconciled | Cell 0 60        | $\top$              | Wand (Cell 0 60) $\top$ |
| Linked list  | read without ownership    | <b>emp</b>       | Cell 0 99           | $\top$                  |

## 9 Monad Laws

Every monad must satisfy three equational laws that together guarantee sequential composition behaves predictably:

- **Left identity:**  $\text{pure } x \gg= f = f \ x$ . Wrapping a value in pure and immediately binding it is the same as applying  $f$  directly; pure introduces no effect of its own.
- **Right identity:**  $e \gg= \text{pure} = e$ . Binding a computation to pure is the same as the computation itself; pure consumes no observable state.
- **Associativity:**  $(e \gg= f) \gg= g = e \gg= (\lambda x \rightarrow f \ x \gg= g)$ . Re-parenthesising a chain of binds does not change its meaning; sequential composition is order-preserving but grouping-independent.

When these laws hold, programmers can freely refactor monadic chains without changing the observable semantics: extracting sub-expressions, fusing steps, or reordering **do**-blocks are all valid.

*Scope of the laws: the (ret, post, future) triple, not pre.* We state upfront, rather than only at the end of this section, exactly what is and is not proved. Pledge’s four fields do not *all* satisfy the monad laws under ordinary equality: pre follows the Hoare-rule discipline of Equation (4), which deliberately deviates from left-identity when a precondition violation is detected (a violation must

remain visible after further binding, not be silently absorbed). What we prove below is that the triple  $(ret, post, future)$ , i.e.  $\text{Pledge}$  values compared *ignoring* the  $pre$  field, satisfies the three laws exactly. Equivalently:  $\text{Pledge}$  is a lawful  $\text{Monad}$  once  $\text{Eq}$  (or the informal notion of “the same specification”) is defined to ignore  $pre$ , and  $pre$  is carried alongside it as a separate, non-monadic contract-checking annotation, analogous to a writer-monad log that is inspected but not required to respect the monad laws on its own. We take this position, rather than trying to force  $pre$  into the same equality, because absorbing precondition violations under left-identity would make  $\text{Pledge}$  silently forget a detected contract violation whenever it is immediately rebound, which is the opposite of what a specification-checking tool should do.

Because  $future$  has type  $a \rightarrow \text{eff}$ , the laws are stated in terms of  $\text{evalFuture } e = \text{future } e (ret \ e)$ , written  $\widehat{future}(e)$  for brevity. The laws hold for the triple  $(ret, post, \widehat{future})$  assuming  $\text{Composable}$  forms a monoid under  $\diamond$  with unit  $\text{empty}$ , and  $\text{universe}$  is the unit for  $(\wedge \setminus)$ .

*Left identity.*  $\text{pure } x \gg= f = f \ x.$

*ret:*  $ret(f \ x). \checkmark$  *post:*  $\varepsilon <> post(f \ x) = post(f \ x). \checkmark$

For the future, let  $fe = f(\text{ret}(\text{pure } x)) = f \ x$ . Since  $\widehat{future}(\text{pure } x) = \neg\emptyset$ , subtraction’s  $\Sigma^*$ -base-case gives  $post(fe) \setminus \neg\emptyset = \varepsilon$  directly from the  $\text{Composable}$  instance’s second base clause (§4); the identity used here is exactly  $r_1 \setminus \neg\emptyset = \varepsilon$  read with  $r_1$  replaced by  $post(fe)$ , i.e. *any* postcondition, once compared against the universal obligation, discharges completely, because there is nothing left to name. Hence:

$$\begin{aligned} \widehat{future}(\text{pure } x \gg= f) &= (post(fe) \setminus \widehat{future}(\text{pure } x)) \wedge \widehat{future}(fe) \\ &= (post(f \ x) \setminus \neg\emptyset) \wedge \widehat{future}(f \ x) = \varepsilon \wedge \widehat{future}(f \ x) = \widehat{future}(f \ x). \checkmark \end{aligned}$$

*Right identity.*  $e \gg= \text{pure } = e.$

*ret:*  $ret(e). \checkmark$  *post:*  $post(e) <> \varepsilon = post(e). \checkmark$

Let  $fe = \text{pure}(\text{ret}(e))$ , so  $post(fe) = \varepsilon$  and  $\widehat{future}(fe) = \neg\emptyset$ . Then, using the other base case  $\varepsilon \setminus r_2 = r_2$ :

$$\widehat{future}(e \gg= \text{pure}) = (\varepsilon \setminus \widehat{future}(e)) \wedge \neg\emptyset = \widehat{future}(e) \wedge \neg\emptyset = \widehat{future}(e). \checkmark$$

*Associativity.* Re-parenthesising  $(e \gg= f) \gg= g$  as  $e \gg= (\lambda x \rightarrow f \ x \gg= g)$  must not change the specification.

*ret* and *post:* both sides yield  $ret(g \ r_2)$  and  $post(e) <> post(f \ r_1) <> post(g \ r_2)$ , following from associativity of  $\diamond$ .  $\checkmark$

For the future, let  $r_1 = ret(e)$ ,  $r_2 = ret(f \ r_1)$ . Unfolding the bind formula (Equation (3)) on both sides and applying associativity of  $\wedge$ , both reduce to the same expression:

$$(post(g \ r_2) \setminus (post(f \ r_1) \setminus \widehat{future}(e))) \wedge \widehat{future}(f \ r_1) \wedge \widehat{future}(g \ r_2). \checkmark$$

*Precondition and the monad-law status of pre, revisited.* Returning to the point flagged at the start of this section: the residual  $post(e) \setminus pre(e_2)$  of Equation (4) captures which part of  $e_2$ ’s precondition  $e$ ’s postcondition has not discharged; full coverage gives the standard Hoare sequencing rule, and absent coverage collapses to  $\emptyset$ , flagging the violation immediately and *persistently* through further binds, which is the behaviour we want from a contract checker and which is incompatible with unrestricted left-identity on  $pre$ . In practice, a programmer inspects  $pre$  and  $\text{evalFuture}$  after constructing the specification term, and both fields equal  $\text{universe}$  if and only if the composed program is contract-correct (Theorem 4.3 for the RE instance).

## 10 Related Work

*Future conditions and temporal verification.* The proposal this paper implements is due to Song et al. [2025], who introduce future conditions as a first-class extension to pre/post contracts and formalise the notion that an operation may discharge temporal obligations onto its continuation, at the level of a formal system rather than a runnable library. Song et al. [2024] apply a related regular-expression encoding of temporal properties in ProveNFix, to guide automated program repair rather than to specify obligations directly. Pledge’s relationship to both is one of realisation and extension: we take the future-condition idea of Song et al. [2025] and give it a concrete monadic interface in Haskell, a derivative-based subtraction operator that implements their discharge relation algorithmically (§2.2), a proof that this operator is sound with respect to trace membership (§4.1), and four Composable instances — Presburger-guarded traces, semiring weights, and separation-logic heaps, alongside plain traces — that neither prior paper instantiates. Where Song et al. [2024] use temporal specifications to drive automated repair, Pledge is oriented toward identifying violations through annotation inspection, prior to any effectful execution, leaving repair to the programmer or to a separate tool.

*Pre/post-condition contracts in Haskell.* Liquid Haskell [Vazou et al. 2014] expresses pre/post-conditions as refinement types verified by an SMT solver. Findler and Felleisen [Findler and Felleisen 2002] establish a runtime contract discipline for higher-order functions. QuickCheck [Claessen and Hughes 2000] supports conditional properties via the implication combinator `==>`, which restricts a property to inputs satisfying a given precondition. Swierstra and Baanen [Swierstra and Baanen 2019] embed Hoare triples as an indexed monad, propagating pre/post annotations through monadic bind analogously to Pledge. Pledge complements these by adding a *temporal* dimension through the future condition, without requiring type-system extensions.

*Parameterised and graded monads.* Atkey [2009] introduces parameterised monads, written  $T(S_1, S_2, A)$ , for Hoare-logic-style pre/post state tracking. Katsumata [2014] generalises this to graded monads over a monoid; Orchard and Yoshida [2016] show effect systems and session types are instances. Pledge is related but orthogonal: rather than indexing by effect *type* at the type level, we carry an explicit obligation as future  $a \rightarrow \text{eff}$  at the value level, preserving compatibility with the standard Monad class and `do`-notation, in the specific, narrower sense of §3.5: composing specifications, not composing specifications together with the real effects they describe.

*Resource management in Haskell.* The bracket combinator lexically scopes cleanup but cannot propagate obligations beyond the lexical scope. The ResourceT [Snoyman 2011] library threads a finaliser queue through every bind, discharging cleanup when the block exits. Both Acquire and Managed generalise this to an applicative/monadic interface. All three handle the *cleanup* half of a resource obligation but cannot express *protocol* or *quantitative* obligations, which require explicit future-condition propagation across bind steps.

*Linear types.* Linear Haskell [Bernardy et al. 2018] introduces multiplicity annotations enforced by GHC since version 9.0; linear-base [Tweag and contributors 2021] provides a RIO monad with linear file handles, statically guaranteeing every opened handle is consumed exactly once. Linear types enforce *structural* obligations (use exactly once), whereas future conditions enforce *temporal* obligations (use in a specified order, or with a specified probability). A hybrid combining linear types for uniqueness with Pledge for temporal ordering could achieve stronger guarantees than either alone.

*Effect systems and algebraic effect handlers.* Algebraic effect handlers [Pretnar 2015] decompose effectful computations into an operation set and a separate handler; resource lifecycle obligations are

typically encoded in the handler’s state machine. Polysemy’s Scoped effect attaches resource creation and cleanup to a lexical scope threaded through the effect row. Pledge takes the complementary route described in §3.5: obligations are values in the annotation fields of a pure specification term, checked as a shadow computation, rather than a handler intercepting a live effect row.

*Session types.* Session types [Gay and Hole 2005; Honda 1993] enforce communication protocols at the type level. The `sessiontypes` library [van Walree 2017] realises this via an indexed monad  $\text{STTerm } m \rightarrow s' \rightarrow a$ ; the type checker rejects programs that violate the protocol order. Pledge is more lightweight: it operates within the standard `Monad` class, admits quantitative and heap-ownership obligations that session types do not, and is applicable beyond message-passing.

*Typestate.* Typestate systems [Bierhoff and Aldrich 2007; Strom and Yemini 1986] encode resource lifecycle protocols in the type system. Brady’s ST monad encoding in Idris [Brady 2013] demonstrates full compile-time enforcement in a dependently typed language. Pledge achieves comparable expressiveness at the library level using standard Haskell type classes, at the cost of moving checking to the point where the specification term’s residual annotations are inspected, rather than to type-inference time: violations surface as non-trivial residual annotations in pre or future, which the programmer evaluates before executing effects rather than relying on the type checker to reject programs.

*Runtime monitoring.* Runtime verification [Leucker and Schallhart 2009] checks temporal properties by instrumenting a concrete execution. Pledge propagates obligations in the specification layer: residuals surface as soon as annotations are composed, prior to any effectful run, without requiring a concrete execution, at the cost — made explicit in §3.5 and §11 — of not itself catching divergence between the specification and the real code it shadows.

*Brzowski derivatives.* Derivatives of regular expressions [Brzowski 1964] have found application in parsing [Might et al. 2011; Owens et al. 2009] and model checking. Equation (1) extends this to complement via  $\partial_a(\neg r) = \neg(\partial_a r)$ ; our subtraction operator is a novel application to future-condition propagation in a monadic setting, made precise by the soundness theorem of §4.1.

## 11 Discussion and Future Work

*Decidability.* Equality of normalised regular expressions is decidable, so the fixed-point computation in subtraction terminates whenever the derivative sequence is finite, which holds for the regular languages used in our examples. Extending to context-free or  $\omega$ -regular specifications remains future work.

*Precondition monad laws.* As discussed at the start of §9, the `pre` field satisfies the Hoare-rule property rather than strict monad-law equality. A natural refinement is to make `pre` a separate, non-monadic annotation computed by a static analysis pass over the Pledge term, separating the behavioural monad from the precondition contract checker; we have not implemented this refactoring, and its main open question is whether the resulting two-pass design still produces the same immediate, per-bind violation signal that the current design gives.

*Closing the gap between the specification and real effects.* §3.5 describes the shadow-specification pattern as the paper’s intended use, and is explicit that Pledge itself does not check that the shadow specification and the real IO computation stay in sync, nor does it intervene if an asynchronous exception causes the real code to diverge from its specification mid-run. Lifting Pledge into an effect-handler framework (`effectful` or `polysemy`), so that a Pledge annotation is attached directly to each operation’s real effect handler and updated automatically as the effect executes, would close this gap and remove the possibility of drift between the two. A lighter-weight intermediate step,

requiring no new effect-handler integration, is to pair Pledge specifications with bracket or mask at the call sites already identified by the specification's primitives, so that at least the obligations Pledge names are also runtime-enforced at those points; we regard this as the most promising near-term direction and leave it for future work.

*Closed-world alphabet assumption.* The `firstWith` function uses  $\Sigma_0 = \text{atoms}(r_1) \cup \text{atoms}(r_2)$  as its working alphabet. This is sound under the *closed-world assumption* that every relevant event is mentioned in the two operands. A fully open-world implementation would accept an explicit  $\Sigma$  declaration, enabling complement over an unrestricted domain.

*Connection to indexed and graded monads.* Because `future :: a → eff` indexes the obligation by the return value, Pledge echoes the indexed/graded-monad literature where types are indexed by computational effects; here the index is a return value instead of an effect row.

*Toward solver-backed SL discharge.* §7 is explicit that the SL instance's subtraction records the magic wand symbolically, without consulting a heap model or invoking a solver, and §4.1 notes that Theorem 4.3's style of guarantee therefore does not currently extend to it. Wiring the residual SL predicates into an existing decision procedure (Z3 via `sbv`, or a dedicated separation-logic prover such as `Smallfoot`) so that `Wand` nodes are checked for satisfiability rather than merely recorded is the concrete, scoped next step toward closing this gap, and toward a soundness statement for SL analogous to Theorem 4.3.

*Parallel composition.* The Pledge monad is inherently sequential; concurrent programs require a parallel combinator  $e_1 \parallel e_2$ . In the RE instance this corresponds to the shuffle product  $r_1 \sqcup\sqcup r_2$ ; the SL instance admits a parallel connective via the concurrent separation logic frame rule [O'Hearn 2007]. Extending `Composable` with a parallel operation is a natural next step.

*Quantitative future conditions.* A natural further direction is to connect WRE to probabilistic temporal logics (PCTL) or quantitative Kleene algebras; in the SL instance, a quantitative analogue could express amortised resource bounds or expected heap usage.

## 12 Conclusion

We have presented *Pledge*, a Haskell library that realises Song et al. [2025]'s proposal of *future conditions* as a concrete monad: annotations that constrain what the remainder of a program must accomplish, checked as a pure specification before, or independently of, any real effectful execution.

The central insight is that trace-based, arithmetic, quantitative, and heap-ownership obligations are structurally identical at the level of a five-operation `Composable` algebra. All four instances (RE, GRE, WRE, SL) share a single monadic bind formula, and the diagnostic signal is uniform: a non-trivial residual in `pre` or `future` identifies exactly which obligation remains unmet and why, a diagnostic we prove sound for the RE instance (Theorem 4.3).

Pledge requires no language extensions beyond standard type classes and composes via a standard Monad instance, letting a specification term be built with ordinary `do`-notation; as discussed in §3.5, this composition operates on the pure specification layer itself; connecting that layer to a real, running effectful computation is the shadow-specification pattern we describe, and tightening that connection is the paper's main direction for future work, alongside parallel composition (shuffle product in the RE instance; frame rule in the SL instance) and solver-backed residual discharge for the SL instance.

## References

- Robert Atkey. 2009. Parameterised Notions of Computation. *Journal of Functional Programming* 19, 3–4 (2009), 335–376. doi:10.1017/S095679680900728X
- Jean-Philippe Bernardy, Mathieu Boespflug, Ryan R. Newton, Simon Peyton Jones, and Arnaud Spiwack. 2018. Linear Haskell: Practical Linearity in a Higher-Order Polymorphic Language. *Proceedings of the ACM on Programming Languages* 2, POPL, Article 5 (2018), 29 pages. doi:10.1145/3158093
- Kevin Bierhoff and Jonathan Aldrich. 2007. Modular Typestate Checking of Aliased Objects. In *Proceedings of the 22nd Annual ACM SIGPLAN Conference on Object-Oriented Programming Systems and Applications (OOPSLA)*. 301–320. doi:10.1145/1297027.1297050
- Edwin Brady. 2013. Idris, a General-Purpose Dependently Typed Programming Language: Design and Implementation. *Journal of Functional Programming* 23, 5 (2013), 552–593. doi:10.1017/S095679681300018X
- Janusz A. Brzozowski. 1964. Derivatives of Regular Expressions. *J. ACM* 11, 4 (1964), 481–494. doi:10.1145/321239.321249
- Koen Claessen and John Hughes. 2000. QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs. In *Proceedings of the 5th ACM SIGPLAN International Conference on Functional Programming (ICFP)*. 268–279. doi:10.1145/351240.351266
- Robert Bruce Findler and Matthias Felleisen. 2002. Contracts for Higher-Order Functions. In *Proceedings of the 7th ACM SIGPLAN International Conference on Functional Programming (ICFP)*. 48–59. doi:10.1145/581478.581484
- Simon J. Gay and Malcolm Hole. 2005. Subtyping for Session Types in the Pi Calculus. *Acta Informatica* 42, 2-3 (2005), 191–225. doi:10.1007/s00236-005-0177-z
- Kohei Honda. 1993. Types for Dyadic Interaction. In *Proceedings of the 4th International Conference on Concurrency Theory (CONCUR) (Lecture Notes in Computer Science, Vol. 715)*. 509–523. doi:10.1007/3-540-57208-2\_35
- Shin-ya Katsumata. 2014. Parametric Effect Monads and Semantics of Effect Systems. *ACM SIGPLAN Notices* 49, 1 (2014), 633–646. doi:10.1145/2578855.2535846
- Martin Leucker and Christian Schallhart. 2009. A Brief Account of Runtime Verification. *Journal of Logic and Algebraic Programming* 78, 5 (2009), 293–303. doi:10.1016/j.jlap.2008.08.004
- Matthew Might, David Darais, and Daniel Spiewak. 2011. Parsing with Derivatives: A Functional Pearl. In *Proceedings of the 16th ACM SIGPLAN International Conference on Functional Programming (ICFP)*. 189–195. doi:10.1145/2034773.2034801
- Peter W. O’Hearn. 2007. Resources, Concurrency, and Local Reasoning. *Theoretical Computer Science* 375, 1–3 (2007), 271–307. doi:10.1016/j.tcs.2006.12.035
- Dominic Orchard and Nobuko Yoshida. 2016. Effects as Sessions, Sessions as Effects. In *Proceedings of the 43rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. 568–581. doi:10.1145/2837614.2837634
- Scott Owens, John Reppy, and Aaron Turon. 2009. Regular-expression Derivatives Re-examined. *Journal of Functional Programming* 19, 2 (2009), 173–190. doi:10.1017/S0956796808007090
- Matija Pretnar. 2015. An Introduction to Algebraic Effects and Handlers. Invited tutorial paper. *Electronic Notes in Theoretical Computer Science* 319 (2015), 19–35. doi:10.1016/j.entcs.2015.12.003
- Michael Snoyman. 2011. conduit: Streaming Data Processing. <https://hackage.haskell.org/package/conduit>. Haskell package on Hackage; introduces the ResourceT transformer.
- Yahui Song, Darius Foo, and Wei-Ngan Chin. 2025. Specifying and Verifying Future Conditions. In *Static Analysis — 32nd International Symposium, SAS 2025 (Lecture Notes in Computer Science, Vol. 16100)*. Springer, 90–112. doi:10.1007/978-3-032-07106-4\_5
- Yahui Song, Xiang Gao, Wenhua Li, Wei-Ngan Chin, and Abhik Roychoudhury. 2024. ProveNFix: Temporal Property-Guided Program Repair. *Proceedings of the ACM on Software Engineering* 1, FSE, Article 11 (2024), 23 pages. doi:10.1145/3643737
- Robert E. Strom and Shaula Yemini. 1986. Typestate: A Programming Language Concept for Enhancing Software Reliability. *IEEE Transactions on Software Engineering* 12, 1 (1986), 157–171. doi:10.1109/TSE.1986.6312929
- Wouter Swierstra and Tim Baanen. 2019. A Predicate Transformer Semantics for Effects (Functional Pearl). *Proceedings of the ACM on Programming Languages* 3, ICFP, Article 103 (2019), 26 pages. doi:10.1145/3341707
- Tweag and contributors. 2021. linear-base: Standard Library for Linear Types in Haskell. <https://github.com/tweag/linear-base>.
- Ferdinand van Walree. 2017. sessiontypes: Session Types for Haskell. <https://hackage.haskell.org/package/sessiontypes>.
- Niki Vazou, Eric L. Seidel, Ranjit Jhala, Dimitrios Vytiniotis, and Simon Peyton Jones. 2014. Refinement Types for Haskell. In *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming (ICFP)*. 269–282. doi:10.1145/2628136.2628161